



Session 01:

Introduction to Git & GitHub

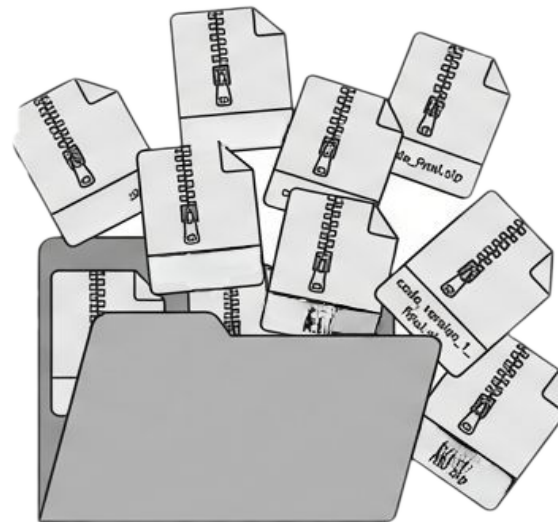


1. Hiểu và thiết lập được hệ thống quản lý mã nguồn (VCS)
2. Hiểu cơ chế hoạt động và sử dụng được các thao tác Git
3. Áp dụng được kỹ thuật làm việc với nhánh trên Git
4. Biết cách xử lý xung đột mã nguồn
5. Sử dụng GitHub để lưu trữ mã nguồn, làm việc nhóm hiệu quả

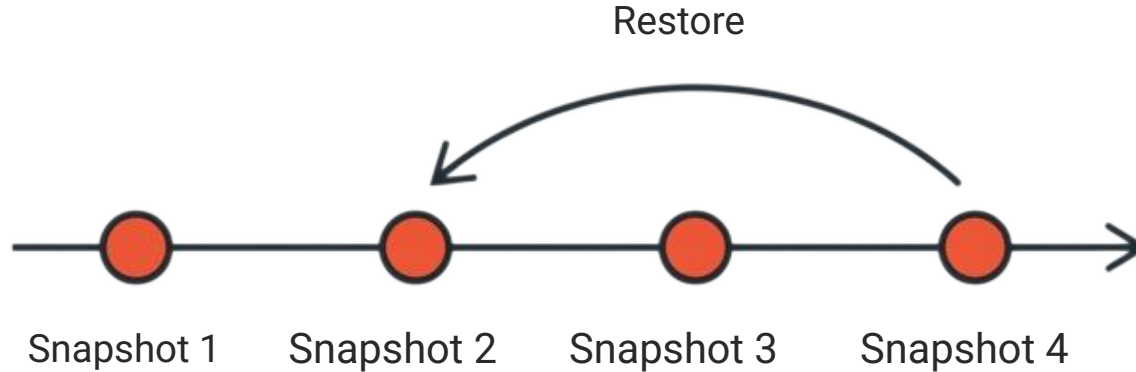
1. Khái niệm về VCS

Tại sao cần VCS?

- **Vấn đề:** Quản lý thủ công gây nhiều loạn file
- **Rủi ro:** Dễ mất dữ liệu, khó khôi phục phiên bản cũ
- **Hậu quả:** Khó làm việc nhóm hiệu quả



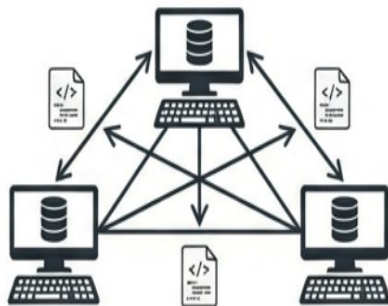
Version Control System (VCS)



- **Khái niệm:** Hệ thống ghi nhận mọi thay đổi của file
- **Cơ chế:** Lưu trữ dưới dạng các Snapshot
- **Lợi ích:** Quay lại bất kỳ thời điểm, phiên bản nào của mã nguồn trong quá khứ

Phân Loại VCS

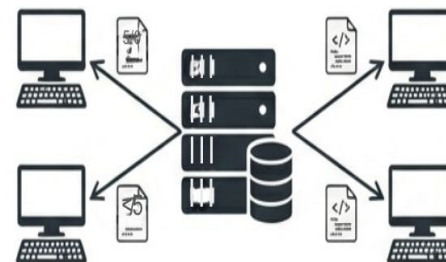
Centralized (SVN)



Code tập trung tại 1 máy chủ

****Warning****: Mất kết nối = Không thể làm việc

Distributed (Git)



Mỗi máy cá nhân là một kho lưu trữ đầy đủ

Best Practice*: An toàn, linh hoạt, hỗ trợ Offline

2. Giới Thiệu Về Git



- **Tác giả:** Linus Torvalds (Cha đẻ Linux)
- **Tốc độ:** Xử lý cực nhanh với dự án lớn
- **Tư duy:** Non-linear development (Phát triển phi tuyến tính)

Git vs GitHub



Là công cụ được cài đặt trên máy tính
Dùng để quản lý phiên bản nội bộ (Local)



Là dịch vụ Hosting trên nền tảng Cloud
Nơi lưu trữ và chia sẻ code với cộng đồng

Cấu Hình Định Danh (Identity)

- Thiết lập **tên** và **email** cho mọi **Commit**
- Kiểm tra lại bằng lệnh: `git config --list`



```
$ git config --global user.name "Your Name"  
$ git config --global user.email "email.gmail.com"
```


Quy Trình 3 Khu Vực

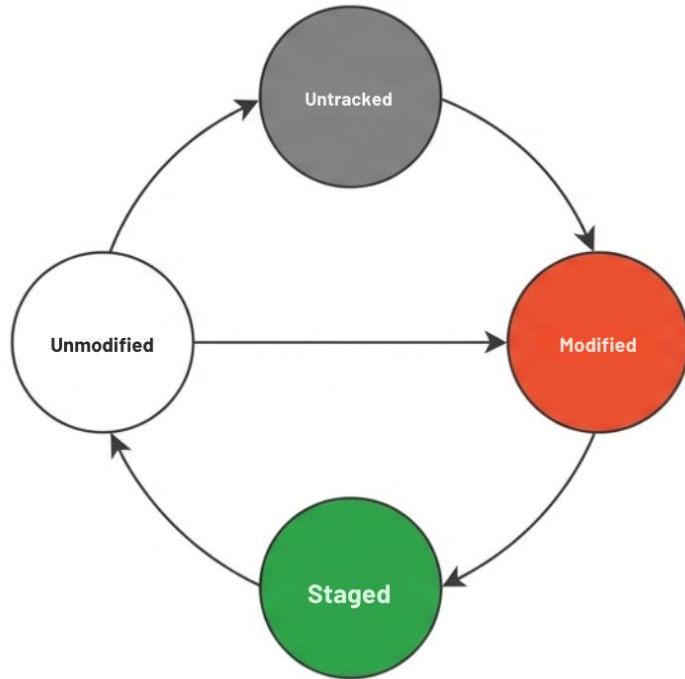


Working Directory: Nơi file đang được chỉnh sửa

Staging Area: Khu vực chọn lọc thay đổi (Nháp)

Local Repo: Nơi lưu trữ chính thức các phiên bản

Vòng Đời Của File



Untracked: File mới, Git chưa theo dõi

Modified: File đã sửa, chưa đưa vào Staging

Staged: File đã sẵn sàng để Commit

Unmodified: File an toàn, chưa có thay đổi mới

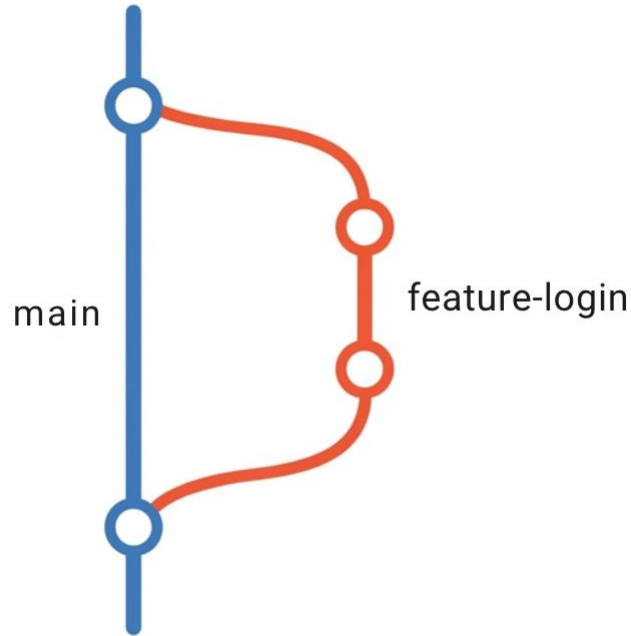
Các Lệnh Thao Tác Cốt Lõi

- **Khởi tạo:** git init (Tạo kho chứa)
- **Kiểm tra:** git status (Xem trạng thái file)
- **Lưu trữ:** Add vào Staging -> Commit vào Repo

```
$ git add .
```

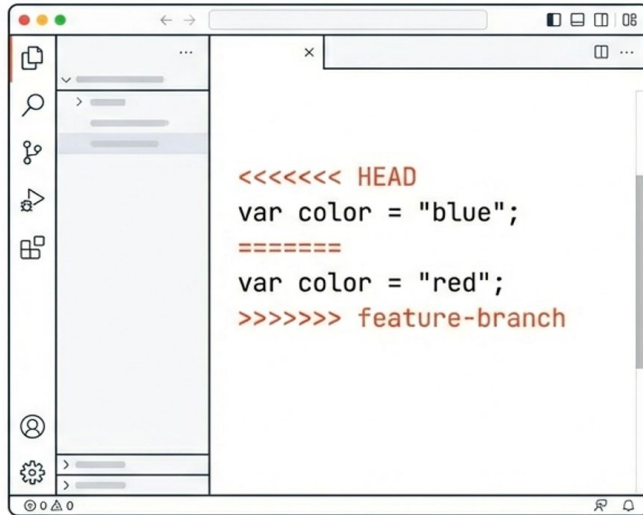
```
$ git commit -m "Ghi chú về thay đổi"
```

3. Tư Duy Làm Việc Theo Nhánh



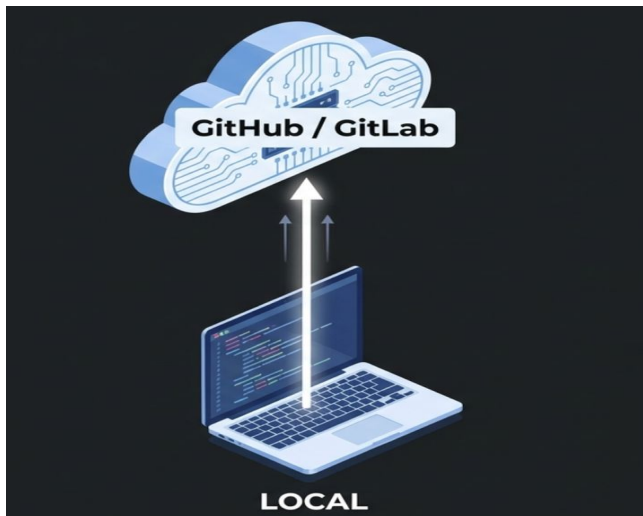
- **Khái niệm:** Nhánh là con đường phát triển tách biệt
- **Mục đích:** Phát triển tính năng (Feature) hoặc sửa lỗi (Bugfix)
- **Lợi ích:** Không làm hỏng code ổn định ở nhánh main.

4. Xử Lý Xung Đột (Conflict)



- **Nguyên nhân:** 2 nhánh cùng sửa một dòng code
- **Xử lý:** Mở file lỗi -> Chọn code cần giữ (Current/Incoming)
- **Kết thúc:** git add và commit lại file đã sửa

5. Quản trị mã nguồn tập trung với Remote Repository

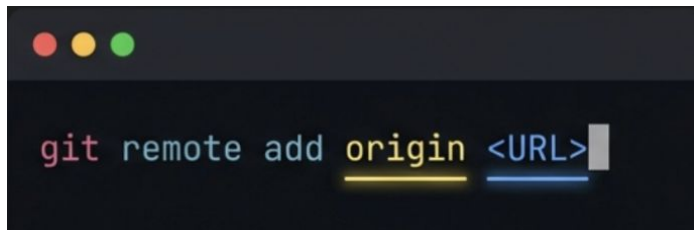


- Lưu trữ mã nguồn trên Server tập trung.
- Nền tảng phổ biến: GitHub, GitLab
- Nơi chia sẻ mã nguồn tập trung

Thiết lập Kết nối

Liên kết Local Repository với Remote Server

- **origin:** Tên gọi nhớ mặc định
- **<URL>:** Đường dẫn kho trên GitHub



```
git remote add origin <URL>
```

Đồng bộ hóa dữ liệu (Pushing changes)



```
git push origin <nhánh>
```

Chuyển các bản Commit từ Local lên Remote Repository.

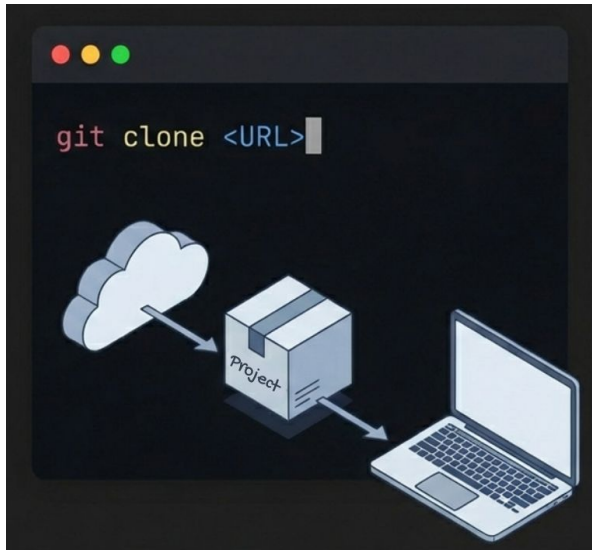
Cập nhật mã nguồn (Pulling changes)



```
git pull origin <nhánh>
```

Cập nhật các thay đổi mới nhất từ Remote vào Local.

Khởi tạo dự án từ Remote (Git Clone)



- Sao chép toàn bộ dự án về máy
- Tự động thiết lập kết nối remote

Lưu ý: Chỉ thực hiện 1 lần đầu tiên

- ❑ Hiểu được khái niệm VCS và tầm quan trọng của việc lưu vết lịch sử mã nguồn trong phát triển phần mềm.
- ❑ Biết cách khởi tạo một kho lưu trữ mới và kiểm tra tình trạng dự án
- ❑ Hiểu được bản chất và vai trò của Remote Repository trong việc quản lý mã nguồn tập trung.
- ❑ Nắm vững quy trình đẩy mã nguồn (Pushing) để đồng bộ các bản commit
- ❑ Hiểu cách thức cập nhật mã nguồn (Pulling)
- ❑ Sử dụng thành thạo kỹ thuật Clone để khởi tạo môi trường làm việc từ một dự án có sẵn trên Remote.



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN